

Malware-JEPA: A Joint-Embedding Predictive Architecture on Heterogeneous Behavior Graphs for Malware Family Classification

Andrés David Montenegro Alvarado[†], Nelson Enrique Vera Parra¹

Universidad Distrital Francisco José de Caldas, Faculty of Engineering, GICOG Research Group, Bogotá, Colombia

Abstract

Malware family classification is bottlenecked by costly and unstable labels, even as dynamic analysis produces large volumes of unlabeled execution traces that admit a natural representation as heterogeneous behavior graphs. Self-supervised learning on such graphs remains difficult: reconstructive pretext tasks overfit low-level input details, while contrastive objectives depend on augmentations that distort typed relational semantics. We propose Malware-JEPA, a Joint-Embedding Predictive Architecture for self-supervised representation learning on heterogeneous malware behavior graphs. Malware-JEPA partitions each behavior graph into context-target sub-graphs through attention-guided adaptive partitioning, encodes typed relational interactions with a Heterogeneous Graph Transformer, and predicts target subgraph representations in latent space, sidestepping both high-dimensional reconstruction and augmentation engineering. On a large-scale heterogeneous behavior graph dataset derived from ACT-KingKong, linear evaluation of the pretrained representations matches a fully supervised HGT when both are trained with only 40% of the labels, and fine-tuning the pretrained encoder with 70% of the labels reaches the fully supervised baseline trained on the complete labeled set. These results show that predictive latent-space pretraining provides an effective path beyond the supervision bottleneck of malware family classification, and establish JEPA as a paradigm for self-supervised learning on relationally rich heterogeneous behavior graphs.

Keywords: Malware family classification; Heterogeneous graph neural networks; Self-supervised learning; Graph representation learning; Joint-embedding predictive architecture

1. Introduction

Malicious software (malware) is the primary source of threat to information systems and their users [1]. A malware sample is designed to compromise the resources of a host system and execute actions that materialize a malicious intent through interactions with operating system

[†]Corresponding author: Andrés David Montenegro Alvarado (Email: admontenegro@udistrital.edu.co; ORCID: 0009-0006-3259-7915)

¹ORCID: 0000-0002-5159-9207

components, with objectives ranging from exfiltration of sensitive information to disruption of critical infrastructure [1, 2, 3].

The scale of the problem is reflected in the roughly 450,000 new samples reported daily by AV-TEST [4], driven both by modular recombination of functional components that implement previously observed malicious operations [3] and by evasion techniques that alter the observable surface of malware without changing its underlying functionality [5, 6]. Such techniques are symptomatic of adversarial dynamics in which evasion is engineered to frustrate automated analysis [5]. These dynamics introduce structural difficulties for labeling at scale, including antivirus signatures that are frequently incomplete or incorrect, different vendors using different names for the same family, and antivirus-consensus labeling that introduces bias by excluding the hardest-to-classify samples [7]. Despite these difficulties, assigning a sample to a known malware family enables analysts to anticipate the behaviors it is likely to execute, and provides a critical input for triage and remediation tasks [8].

Operationalizing this family classification at scale requires a representation that captures the behavioral interactions characteristic of each one. Heterogeneous graphs encode interactions among multiple types of entities and relations, with structural and semantic dependencies determined by the relational context [9, 10, 11]. This relational structure is particularly relevant in cybersecurity, where malware execution traces induce heterogeneous graphs composed of entities and behavioral events [12, 13, 14].

Learning over such graphs, however, faces a supervision bottleneck. Manual labeling of malware families is an extremely costly task that requires highly specialized expert knowledge [8, 15, 7]. While assigning a sample to a family does not always demand exhaustive analysis, when reverse engineering is required, the analysis of a single binary can take from several hours to weeks of expert work [15]. As a consequence, the large-scale generation of reliable labels that describe the underlying behavior of malware rapidly becomes infeasible. This limitation is reflected in the notable fact that, to date, no dataset exists whose samples have been labeled entirely through manual analysis [7].

Given this scalability barrier, self-supervised learning emerges as a natural alternative. Cybersecurity firms can receive more than one million unique malware samples per day [16], providing an abundant data source from which to learn representations of malicious behavior without large-scale human annotation as the primary source of supervision.

Generative methods based on reconstruction learn representations by optimizing the recovery of partially hidden graph structures or attributes, and their effectiveness critically depends on the design of the reconstructive objective, the masking strategy, and the discriminability of the input features [17, 18, 19, 20]. Under this paradigm, latent representations must remain descriptive enough of the full observed data space to support the complex distribution estimation that reconstruction requires, which can hinder the abstraction of high-level semantic regularities, particularly in structurally complex domains [21, 22]. On graphs, this difficulty can be exacerbated by the non-Euclidean nature of the structural space and the topological dependencies inherent among entities [22].

Contrastive methods on graphs learn representations by maximizing the consistency across different perturbed views of the same sample, constructed via augmentations of the graph structure and attributes [23, 24, 25]. Under this approach, the learned representations tend to be invariant to the perturbations induced by augmentations, so learning quality critically depends on those perturbations preserving the intrinsic patterns and relevant semantics of the graph [26].

Consequently, structural or attribute perturbations that do not respect the semantic importance of different graph components can degrade the quality of the learned representations [26]. On heterogeneous graphs, where edge types and relational patterns encode distinct semantics over the interactions among entities [11, 9], this sensitivity intensifies: perturbations applied without regard for such typed semantics can distort the underlying semantic relations of the graph and adversely affect the quality of the learned representations [27].

Joint-Embedding Predictive Architectures (JEPA) formulate self-supervised learning as the prediction of dependencies among latent representations, rather than explicitly reconstructing observations in the input space [28]. By focusing on the prediction of consistent and predictable aspects in the latent space, JEPA favors the learning of more abstract and semantic representations, concentrating on high-level predictive regularities rather than modeling input-specific details [28, 29]. JEPA does not depend on the design of augmentation schemes to induce semantic invariances, thereby avoiding the need to impose additional assumptions about which perturbations preserve meaning or which samples should be considered semantically dissimilar [29, 30].

This paradigm has been extended to the graph domain by Graph-JEPA [30], which predicts the latent representations of masked subgraphs from the representation of a context subgraph. Inspired by this approach, we introduce **Malware-JEPA**, a self-supervised pretraining method that extends Graph-JEPA to the domain of heterogeneous graphs modeling malware behavioral events.

The regularities that distinguish malware families reside in high-level operational patterns that manifest in how multiple behavioral events relate to one another, rather than in superficial details that may vary across samples of the same family without altering the underlying functionality. Through latent-space prediction over heterogeneous subgraphs extracted from execution traces, Malware-JEPA induces representations whose latent structure can be exploited for the taxonomic classification of families.

We summarize our contributions as follows.

- (i) **Malware-JEPA: extension of Graph-JEPA to heterogeneous graphs.** To the best of our knowledge, this is the first work that extends the Graph-JEPA paradigm to the domain of heterogeneous graphs modeling malware behavioral events, articulating a context–target sampling protocol over heterogeneous subgraphs and a predictive objective in abstract representation space adapted to the typed nature of relations among events.
- (ii) **Transferability under a limited-labeling regime.** A linear probe evaluation of the representations induced by Malware-JEPA, without access to family labels during pretraining, matches the performance of the fully supervised Heterogeneous Graph Transformer (HGT) baseline when both are evaluated using only 40% of the labels, demonstrating that the latent space captures linearly decodable structure for the taxonomic classification of malware families.
- (iii) **Label efficiency under fine-tuning.** Under fine-tuning, the representations induced by Malware-JEPA reach 99% of the fully supervised baseline’s performance using 2.5× fewer labels and match this performance using only 70% of the labels.
- (iv) **Advantage of self-supervised pretraining beyond the limited-labeling regime.** Under fine-tuning, we observe that Malware-JEPA achieves higher performance than the supervised baseline at every evaluated labeling fraction, suggesting that self-supervised pretraining in abstract representation space may offer advantages over supervised training beyond the limited-labeling regime.

2. Related Work

Recent work has explored self-supervised representation learning for malware analysis. Two families of objectives dominate this literature: reconstructive and contrastive. Among reconstructive approaches, graph autoencoder methods learn compact embeddings from API-call graphs before downstream classification [31], while Android malware models combine heterogeneous graphs, meta-path semantics, and feature fusion via autoencoders with downstream classifiers [32, 33]. In a different direction, VQ-VAE has also been applied to zero-shot ransomware classification by integrating latent generative modeling with malware knowledge graphs [34]. Across these variants, the pretext signal remains anchored to recovering or compressing input-level information. Such objectives can favor representations that describe the input distribution rather than predict high-level behavioral regularities.

Contrastive approaches instead learn representations by maximizing agreement between multiple views of malware behavior. SCLMD constructs heterogeneous networks from API-call sequences and aligns structural and sequential views through contrastive learning under limited supervision [35]. MINES further extracts API-existence and API-transition features using graph and sequence-level contrastive objectives [36]. FewM-HGCL extends this paradigm to heterogeneous malware graphs by using API attribute masking, interaction enhancement, and meta-path sampling to construct positive and negative graph pairs for few-shot variant detection [37]. More recent work explicitly identifies semantic shift in augmented graphs and semantic sparsity in tail graphs as key limitations of graph contrastive malware detection [38], exposing the dependence of these methods on augmentation schemes that preserve behavioral semantics.

A third paradigm, Joint-Embedding Predictive Architectures, takes a different approach. Rather than reconstructing the input or relying on augmentation-induced invariances, it predicts the latent representations of masked content from visible context [28]. I-JEPA instantiates this idea for images by predicting the representations of target blocks from a single context block, and explicitly identifies the choice of context–target masking as a critical factor for the semantic content of the learned representations [29]. The paradigm has since been extended to relational data: Graph-JEPA [30] predicts the latent representations of randomly sampled target subgraphs from a context subgraph on homogeneous graphs, and concurrent work has applied JEPA pre-training to polymer molecular graphs, improving downstream performance under scarce labels [39]. These graph-level instances, however, target homogeneous graphs and rely on uniformly random subgraph partitioning, neither of which transfers directly to settings in which edge types encode distinct relational semantics and in which behaviorally coherent partitioning matters for the pretext signal.

Malware-JEPA differs from the preceding paradigms by formulating self-supervised learning on heterogeneous malware behavior graphs as latent-space prediction between context and target subgraphs. Rather than reconstructing graph features or enforcing invariance across augmented graph views, it predicts target representations from contextual behavioral regions, avoiding feature-level reconstruction, negative-pair discrimination, and hand-crafted graph augmentations. Relative to Graph-JEPA, Malware-JEPA extends predictive latent-space learning to typed relational structure through a Heterogeneous Graph Transformer encoder and replaces uniformly random subgraph sampling with attention-guided adaptive partitioning that selects context–target pairs from behaviorally coherent regions of the graph.

3. Background

3.1. Heterogeneous behavior graphs

A directed heterogeneous graph is defined as the tuple

$$G = (V, E, \mathcal{A}, \mathcal{R}, \tau_v, \tau_e), \quad (1)$$

where V denotes the set of nodes, $E \subseteq V \times V$ the set of directed edges, $\mathcal{A}$ the set of entity types, and $\mathcal{R}$ the set of relation types. The functions $\tau_v : V \rightarrow \mathcal{A}$ and $\tau_e : E \rightarrow \mathcal{R}$ assign a semantic type to each node and edge, respectively. The graph is heterogeneous when more than one entity or relation type is present, that is, when $|\mathcal{A}| + |\mathcal{R}| > 2$. For any node $t \in V$, we denote by $\mathcal{N}(t) = \{s \in V : (s, t) \in E\}$ its in-neighborhood, comprising all source nodes that point to t through any relation type.

Each directed edge $e = (s, t) \in E$, where s and t denote the source and target nodes, respectively, induces a meta-relation defined by the triplet

$$(\tau_v(s), \tau_e(e), \tau_v(t)) \in \mathcal{A} \times \mathcal{R} \times \mathcal{A}, \quad (2)$$

which jointly specifies the semantic types of the source and target entities together with the nature of their interaction [9]. This formulation preserves relational semantics explicitly through heterogeneous types of entities and interactions, and has been adopted in prior work on malware variant detection [12].

3.2. Heterogeneous Graph Transformer (HGT)

Heterogeneous Graph Transformer (HGT) [9] is a message-passing architecture for heterogeneous graphs that parameterizes attention and messages *per meta-relation*, removing the need for hand-crafted meta-paths. By coupling the parameters to the meta-relation triplet defined in Eq. (2), HGT maintains type-specific representation spaces for nodes and relations and captures that the semantics of an interaction depends jointly on the entity and relation types involved.

Let $H^{(l)} \in \mathbb{R}^{|V| \times d}$ denote the output of the l -th HGT layer, which also serves as the input to the $(l + 1)$ -th layer. Attention is computed with h heads, each of dimension $d_h = d/h$. For each edge $e = (s, t)$ with $s \in \mathcal{N}(t)$, query and key vectors are produced by node-type-specific linear projections:

$$\begin{aligned} Q^i(t) &= \text{Q-Linear}_{\tau_v(t)}^i(H^{(l-1)}[t]), \\ K^i(s) &= \text{K-Linear}_{\tau_v(s)}^i(H^{(l-1)}[s]), \end{aligned} \quad (3)$$

where $Q^i, K^i \in \mathbb{R}^{d_h}$ for each head $i \in \{1, \dots, h\}$. The per-head similarity between s and t is modulated by a relation-type-specific matrix $W_{\tau_e(e)}^{\text{ATT}} \in \mathbb{R}^{d_h \times d_h}$, which allows the model to capture distinct semantic relations between node pairs of the same type, and by a learnable scalar prior $\mu_{\langle \tau_v(s), \tau_e(e), \tau_v(t) \rangle}$ associated with the meta-relation, which encodes its general significance and serves as an adaptive scaling of the attention [9]. The HGT attention is obtained by concatenating the h per-head scores and applying softmax over the typed neighborhood $\mathcal{N}(t)$:

$$\text{Attention}_{\text{HGT}}(s, e, t) = \text{Softmax}_{\forall s \in \mathcal{N}(t)} \left\|_{i \in [1, h]} \left(\frac{K^i(s) W_{\tau_e(e)}^{\text{ATT}} Q^i(t)^\top}{\sqrt{d_h}} \cdot \mu_{\langle \tau_v(s), \tau_e(e), \tau_v(t) \rangle} \right), \quad (4)$$

where $\parallel$ denotes concatenation across heads.

Messages are computed in parallel to the attention scores and follow the same type-aware design, with the goal of alleviating distributional differences across node and edge types. For each neighbor s , the source representation is first projected into a per-head message space by a node-type-specific operator $\text{M-Linear}_{\tau_v(s)}^i : \mathbb{R}^d \rightarrow \mathbb{R}^{d_h}$, and then transformed by a relation-type-specific matrix $W_{\tau_e(e)}^{\text{MSG}} \in \mathbb{R}^{d_h \times d_h}$ that incorporates the edge dependence. The resulting per-head messages are then weighted by the attention scores and aggregated over $\mathcal{N}(t)$. Because the softmax in Eq. (4) normalizes the attention weights to sum to one over the neighborhood, this aggregation is effectively a weighted average of the source messages [9]:

$$\begin{aligned} \text{Message}^i(s, e, t) &= \text{M-Linear}_{\tau_v(s)}^i \left(H^{(l-1)}[s] \right) W_{\tau_e(e)}^{\text{MSG}}, \\ \tilde{H}^{(l)}[t] &= \parallel_{i=1}^h \sum_{s \in \mathcal{N}(t)} \text{Attention}_{\text{HGT}}^i(s, e, t) \cdot \text{Message}^i(s, e, t). \end{aligned} \quad (5)$$

The aggregated vector $\tilde{H}^{(l)}[t]$ now combines information directed to the target node t from neighbors whose representations belong to type-specific feature distributions, yielding a heterogeneous mixture that no longer lies in the feature space of $\tau_v(t)$.

To map this mixture back to the type-specific distribution of t , a node-type-specific output projection $\text{A-Linear}_{\tau_v(t)} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is applied after an element-wise activation σ , followed by a residual connection from the previous layer:

$$H^{(l)}[t] = \text{A-Linear}_{\tau_v(t)} \left(\sigma \left(\tilde{H}^{(l)}[t] \right) \right) + H^{(l-1)}[t], \quad (6)$$

where $\sigma(\cdot)$ denotes the GELU non-linear activation.

Stacking L such layers allows each node's representation to incorporate information from its L -hop typed neighborhood through type-aware attention, without requiring hand-crafted meta-paths [9]. The output $H^{(L)}$ of the final layer constitutes the node representations consumed by downstream components, making HGT a suitable heterogeneous encoder for Malware-JEPA.

3.3. Joint-Embedding Predictive Architecture (JEPA)

A JEPA architecture [28] consists of three parameterized components: a context encoder Enc_x with parameters θ_x , a target encoder Enc_y with parameters θ_y , and a predictor Pred with parameters ϕ . Given a context input x and a target input y , the two encoders map their inputs into a shared latent space,

$$s_x = \text{Enc}_x(x), \quad s_y = \text{Enc}_y(y), \quad s_x, s_y \in \mathbb{R}^d,$$

while the predictor estimates the target representation conditioned on a latent variable z that identifies which target is being predicted:

$$\tilde{s}_y = \text{Pred}(s_x, z).$$

Training minimizes an energy or distance function $D(\tilde{s}_y, s_y)$ between the predicted representation $\tilde{s}_y$ and the target representation s_y [28]. The parameters of the target encoder are updated by an exponential moving average (EMA) of the context encoder parameters:

$$\theta_y \leftarrow \alpha \theta_y + (1 - \alpha) \theta_x, \quad \alpha \in [0, 1), \quad (7)$$

while θ_x and ϕ are optimized by gradient descent on D . The combination of stop-gradient and EMA breaks the symmetry between the two branches and prevents the representation collapse that would otherwise admit constant trivial solutions [28], allowing predictive learning in latent space without negative samples.

Graph-JEPA [30] instantiates this paradigm when the inputs are subgraphs extracted from a homogeneous graph $G = (V, E)$, that is, a graph without entity or relation types ($|\mathcal{A}| = |\mathcal{R}| = 1$). The node set V is partitioned into p subgraphs $\{G'_1, \dots, G'_p\}$, and each subgraph is encoded into a latent representation through a GNN followed by a permutation-invariant pooling operation, producing subgraph embeddings $\{h_1, \dots, h_p\}$ with $h_i \in \mathbb{R}^d$. At training time, a subset of these subgraph embeddings plays the role of s_x (context) and the remaining ones play the role of $s_y^{(i)}$ (targets) in the JEPA objective [30]. Because subgraph embeddings alone do not identify the position of each subgraph within G , Graph-JEPA additionally associates with each subgraph a positional embedding $z^{(i)}$ that encodes its structural location and serves as the latent variable supplied to the predictor. Malware-JEPA builds on this formulation by instantiating predictive latent-space learning over heterogeneous behavior subgraphs, replacing the homogeneous GNN encoder of Graph-JEPA with HGT (Sec. 3.2) and adapting the subgraph partitioning to the typed relational structure of behavior graphs.

4. Malware-JEPA

4.1. Overview

Let $\mathcal{D} = \{(G_n, y_n)\}_{n=1}^N$ be a dataset of malware samples, where each $G_n = (V_n, E_n, \mathcal{A}, \mathcal{R}, \tau_v, \tau_e)$ is the heterogeneous behavior graph of sample n following the formalism of Eq. (1), and $y_n \in \mathcal{Y} = \{1, \dots, C\}$ denotes its family label among C malware families. The goal is to discriminate families from the structural and relational regularities of behavior graphs. Because family labels are scarce and expensive to obtain, Malware-JEPA decouples representation learning from supervision: a self-supervised pretraining stage uses the unlabeled graphs $\{G_n\}_{n=1}^N$, and a downstream stage trains a supervised classifier on a labeled subset $\mathcal{D}_L \subseteq \mathcal{D}$.

We hypothesize that the structural and relational regularities that distinguish malware families can be captured by predicting the latent representations of behaviorally coherent subgraphs from one another, without reconstructing individual events or contrasting sample pairs—a hypothesis evaluated empirically in Section 5. Operationally, each behavior graph G_n traverses five stages: (i) heterogeneous behavior graph construction from execution traces (Sec. 4.2); (ii) relation-aware partitioning into subgraphs whose communities reflect the typed semantics of the graph (Sec. 4.3); (iii) HGT-based encoding of each subgraph into a fixed-dimensional latent representation (Sec. 4.4); (iv) context-target sampling among subgraph embeddings followed by JEPA-style latent prediction, optimized through a self-supervised objective (Sec. 4.5); and (v) downstream classification on top of the pretrained representations (Sec. 4.6).

Figure 1 summarizes the resulting pipeline and the interaction between its components: heterogeneous graph construction, relation-aware structural partitioning, predictive latent-space encoding, and downstream family classification.

4.2. Behavior graph construction

Each sample in the dataset is associated with an execution trace captured by dynamic analysis in a sandbox. Following heterogeneous formulations previously used for malware vari-

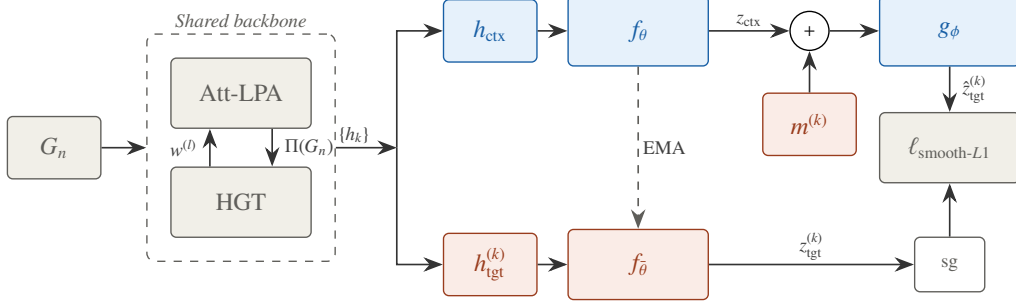


Figure 1: Overview of Malware-JEPA. The HGT encoder plays a dual role: its attention guides Att-LPA to partition the input graph G_n , and its representations seed the JEPA context-target branches that converge in latent space under stop-gradient and EMA.

ant detection [12, 40, 41], every event observed during execution instantiates a typed edge of the behavior graph $G_n = (V_n, E_n, \mathcal{A}, \mathcal{R}, \tau_v, \tau_e)$ defined in Sec. 3.1 as the meta-relation triplet $(\tau_v(s), \tau_e(e), \tau_v(t)) \in \mathcal{A} \times \mathcal{R} \times \mathcal{A}$, where $s, t \in V_n$ are the source and target entities of the event and $e = (s, t) \in E_n$. For instance, the event “the process `server.exe` writes the value `XP.FW.Disable` into the registry key `MICROSOFT\DXDIAG\WINSETTINGS`” is encoded as the typed edge (Process, writes, Registry).

The set of entity types observed during execution comprises eight categories, $\mathcal{A} = \{\text{Process, File, Network, Registry, Memory, Mutex, Attribute, System}\}$, and the relation set $\mathcal{R}$ contains the twelve typed interactions listed in Table 1, which also specifies the admissible source and target entity types of each meta-relation. Among them, Process appears as a valid source type in eleven of the twelve meta-relations of the schema, reflecting its structural role as the entity that mediates most operational interactions in the trace. Each node $v \in V_n$ is associated with an input feature vector that initializes the representation $H^{(0)}[v] \in \mathbb{R}^d$ consumed by the HGT encoder of Sec. 3.2.

Table 1: Admissible meta-relations of the behavior graph. Each row specifies one relation $r \in \mathcal{R}$ with the source and target entity types it accepts from $\mathcal{A}$; a set denotes multiple admissible types.

Relation	Source	Target
connects	Process	Network
creates	Process	{Mutex, System}
deletes	Process	{File, Registry}
fork	Process	Process
listens	Process	Network
modifies	Process	{File, Process, Registry, System}
queries	Process	{Network, Process, System}
reads	Process	{File, Memory, Network}
sends	Process	Network
terminates	Process	Process
triggers	{Process, System}	Attribute
writes	Process	{File, Memory, Registry}

The twelve relations capture semantically distinct operational mechanisms of malware behavior, including process propagation, registry persistence, memory manipulation, network communication, and system resource modification. Edges of different types are therefore not interchangeable from the perspective of representation learning—a premise that motivates the relation-aware partitioning and the type-aware encoding introduced in the following subsections.

4.3. Subgraph context–target sampling

Graph-JEPA constructs context–target pairs from node partitions defined on homogeneous graphs. If adopted directly on behavior graphs, such a topology-only partition risks splitting regions that share relational semantics or grouping interactions of different operational nature. Malware-JEPA therefore replaces the uniform partition with an attention-weighted label propagation procedure, denoted Att-LPA and inspired by SHGP [42], which uses the HGT attention as a relational signal that informs which nodes should group into a community.

Relation-aware attention weights. Let $\alpha_i^{(l)}(e)$ denote the scalar attention score that head i of layer l of the HGT assigns to edge $e \in E_n$, as produced by Eq. (4). Since graphs in the dataset are independent execution traces with no node-to-node or edge-to-edge correspondence, attention scores on individual edges are not directly transferable across graphs. In contrast, relation types provide a shared semantic support across samples. Moreover, the type-specific parameters $W_{\tau_e(e)}^{\text{ATT}}$ and $\mu_{\langle\tau_v(s), \tau_e(e), \tau_v(t)\rangle}$ that generate $\alpha_i^{(l)}$ are shared globally across the dataset. Therefore, aggregating attention scores over edges of the same relation type yields a model-dependent estimate of the expected attention assigned to that relation under the current data distribution.

Concretely, for each relation type $r \in \mathcal{R}$, we define

$$w_r^{(l)} = \frac{1}{h} \sum_{i=1}^h \frac{1}{|S_r^{(\mathcal{B})}|} \sum_{e \in S_r^{(\mathcal{B})}} \alpha_i^{(l)}(e), \quad S_r^{(\mathcal{B})} = \{e \in E^{(\mathcal{B})} : \tau_e(e) = r\}, \quad (8)$$

where $\mathcal{B}$ is the current mini-batch and $E^{(\mathcal{B})} = \bigsqcup_{G_n \in \mathcal{B}} E_n$ denotes the disjoint union of the edge sets of all graphs in the mini-batch. The resulting quantity $w_r^{(l)}$ serves as a batch-level estimate of the attention mass assigned to relation r at layer l . Att-LPA then uses these relation-level weights as a learned structural prior to guide the partitioning process toward behaviorally salient interactions.

Layer-synchronized weighted label propagation. Att-LPA initializes each node with a unique label and iteratively propagates labels over the graph structure [42]. Following the Att-LPA principle, label propagation mirrors the encoder’s attention-based aggregation: at step l , previous labels are aggregated using the relation-level weights $w_r^{(l)}$ extracted from the l -th HGT layer. Thus, labels propagate across typed behavioral relations according to the encoder’s layer-specific attention patterns.

Propagation runs for L steps synchronized with the L layers of the HGT. This ties the depth of the partitioning process to the receptive field of the encoder rather than introducing the number of propagation steps as an independent hyperparameter. After the final step, nodes sharing the same propagated label define the communities used for context–target sampling.

Context–target sampling. After L propagation steps, nodes sharing the same final label define a community, yielding a partition $\Pi(G_n) = \{C_1, \dots, C_p\}$. The number of communities p is not prescribed as a hyperparameter; it emerges from the propagation dynamics and may vary across graphs according to the relation-aware structure induced by the current attention weights.

Given $\Pi(G_n)$, Malware-JEPA uniformly samples one community as context and up to K other communities as targets. Sampling uniformly over communities, rather than over individual nodes, prevents large regions from dominating the pretext task and encourages prediction across distinct relation-aware subgraphs. Unlike random-walk or k -hop sampling, which relies primarily on graph proximity, this procedure constructs prediction units induced by Att-LPA and therefore couples the JEPA pretext to the relational preferences learned by the encoder.

Crucially, the partition is a means rather than an end: unlike SHGP [42], where communities provide pseudo-supervision targets, Malware-JEPA uses them to define the context and target subgraphs over which the JEPA prediction task is constructed. Each selected community is then passed to the encoder as a candidate subgraph in the following stage.

4.4. Encoder and predictor

Subgraph encoding. Before encoding, each community C_k of the partition $\Pi(G_n)$ is expanded with its 1-hop neighbors in G_n , yielding a subgraph $C'_k = (V'_k, E'_k)$ with $C_k \subseteq V'_k$. This expansion introduces a controlled overlap between communities and prevents the loss of neighborhood information induced by strictly disjoint partitions [30]. A single HGT [9], shared between the context and target branches, encodes each expanded subgraph according to the layer operation of Eq. (6), producing one node-level representation $H^{(L)}[v] \in \mathbb{R}^d$ for every $v \in V'_k$. The subgraph embedding $h_k \in \mathbb{R}^d$ is then obtained by mean-pooling these node representations over the nodes contained in the community, $v \in C_k$, so that the 1-hop neighbors enrich the message passing through which the HGT computes each $H^{(L)}[v]$ but do not contribute directly to the pooled summary. Mean pooling is permutation-invariant and yields a fixed-dimensional embedding regardless of community size, which is required because $|C_k|$ varies across communities and across graphs.

Positional encoding. The predictor receives a single context representation but must produce $|\mathcal{T}|$ distinct target estimates, which requires an additional signal that identifies *what* target is being predicted. This signal is the latent variable z of the JEPA formulation (Sec. 3.3), here instantiated as a per-subgraph positional encoding $m^{(k)} \in \mathbb{R}^d$ that captures the typed composition of C'_k . Specifically, two normalized vectors are computed: $\text{PE}_{\text{node}}(C'_k) \in \mathbb{R}^{|\mathcal{A}|}$ contains the relative frequency of each entity type in V'_k , and $\text{PE}_{\text{edge}}(C'_k) \in \mathbb{R}^{|\mathcal{R}|}$ contains the relative frequency of each relation type in E'_k . Both vectors are concatenated and projected to the latent dimension through a learnable matrix $W_{\text{pe}} \in \mathbb{R}^{d \times (|\mathcal{A}| + |\mathcal{R}|)}$:

$$m^{(k)} = W_{\text{pe}} [\text{PE}_{\text{node}}(C'_k) \oplus \text{PE}_{\text{edge}}(C'_k)] \in \mathbb{R}^d. \quad (9)$$

The positional encoding thus preserves global semantic information about the typed composition of each subgraph without exposing its internal connectivity or topology.

Context and target encoders. From the set of subgraph embeddings $\{h_1, \dots, h_p\}$ produced by the encoder, the sampling procedure of Sec. 4.3 designates one embedding $h_{\text{ctx}} = h_c$ as context and a set $\{h_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}}$ of $|\mathcal{T}|$ target embeddings. Their latent representations $z_{\text{ctx}} = f_{\theta}(h_{\text{ctx}})$

and $\{z_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}} = f_{\bar{\theta}}(\{h_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}})$ are then produced by the context encoder f_{θ} and target encoder $f_{\bar{\theta}}$, which together instantiate the JEPA components Enc_x and Enc_y of Sec. 3.3; the parameters $\bar{\theta}$ are updated by an EMA of θ as specified in Sec. 4.5. The two encoders are architecturally asymmetric: f_{θ} is a multilayer perceptron applied to the single context embedding, whereas $f_{\bar{\theta}}$ jointly processes the target embeddings through a Transformer with self-attention [43], allowing the target representations to capture semantic dependencies between distinct target regions of the same behavior graph. This design reflects the asymmetric nature of the JEPA pretext: the context must condense enough information to predict multiple target regions, while the targets benefit from an encoder that can model dependencies among related heterogeneous subgraphs.

Predictor. The predictor g_{ϕ} is a two-layer MLP with GELU activation that estimates the latent representation of each target subgraph from the context representation, conditioned on the corresponding positional encoding through additive composition:

$$\hat{z}_{\text{tgt}}^{(k)} = g_{\phi}(z_{\text{ctx}} + m^{(k)}) \in \mathbb{R}^d, \quad k \in \mathcal{T}. \quad (10)$$

The additive composition $z_{\text{ctx}} + m^{(k)}$ realizes the conditioning of the predictor on the latent variable z of Sec. 3.3: the same context representation z_{ctx} produces a distinct prediction $\hat{z}_{\text{tgt}}^{(k)}$ for each target by shifting it according to the typed signature of the corresponding subgraph. Together, $\{\hat{z}_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}}$ and $\{z_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}}$ form the prediction-target pairs whose alignment is optimized by the self-supervised objective of the following subsection.

4.5. Self-supervised training objective

Loss and asymmetric update. The training objective measures the alignment between predicted and target representations in latent space. Concretely, the loss is the Smooth L1 distance between each prediction $\hat{z}_{\text{tgt}}^{(k)}$ and its target representation $z_{\text{tgt}}^{(k)}$, averaged over the $|\mathcal{T}|$ targets:

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} \ell_{\text{smooth-L1}}(\hat{z}_{\text{tgt}}^{(k)}, \text{sg}(z_{\text{tgt}}^{(k)})), \quad (11)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator that, together with the EMA update of $\bar{\theta}$ (Sec. 3.3), prevents representational collapse, and $\ell_{\text{smooth-L1}}$ is applied element-wise across the d components of the embeddings:

$$\ell_{\text{smooth-L1}}(\hat{z}, z) = \frac{1}{d} \sum_{i=1}^d s_{\beta}(\hat{z}_i, z_i), \quad s_{\beta}(\hat{z}_i, z_i) = \begin{cases} (\hat{z}_i - z_i)^2 / (2\beta), & |\hat{z}_i - z_i| < \beta, \\ |\hat{z}_i - z_i| - \beta/2, & \text{otherwise.} \end{cases} \quad (12)$$

The parameters $(\theta_{\text{HGT}}, \theta, \phi)$ are updated by gradient descent on $\mathcal{L}_{\text{JEPA}}$, while the target encoder parameters $\bar{\theta}$ follow an EMA of θ (Eq. 7).

Training schedule. Training proceeds in two phases. Similarly to the warm-up proposed by SHGP [42], in an *initialization phase* of E_0 epochs the partitions are computed once per graph using classical LPA [44] and held fixed while the HGT is optimized against them, providing a stable structural signal before the partition starts to depend on the learned attention. In the subsequent *adaptive phase*, Att-LPA replaces classical LPA and is recomputed at every step from the current relation weights $\{w^{(l)}\}$, so that the community structure co-evolves with the

encoder. Because Att-LPA itself is non-differentiable and lies outside the gradient path, this co-evolution leaves the structural update decoupled from the continuous optimization of the encoder. Algorithm 1 summarizes one training step of the adaptive phase.

Algorithm 1 One Malware-JEPA training step on a graph G_n of the mini-batch $\mathcal{B}$ (adaptive phase).

Require: Behavior graph $G_n \in \mathcal{B}$; parameters $(\theta_{\text{HGT}}, \theta, \bar{\theta}, \phi)$; number of targets K ; EMA coefficient α .

- 1: Extract relation-level weights $\{w^{(l)}\}_{l=1}^L$ from the HGT attention over $\mathcal{B}$ ▷ Sec. 4.3
 - 2: Compute partition $\Pi(G_n) \leftarrow \text{Att-LPA}(G_n, \{w^{(l)}\})$ ▷ Sec. 4.3
 - 3: $h_k \leftarrow \text{ENCODE}(C_k) \quad \forall k \in \{1, \dots, p\}$ ▷ Sec. 4.4
 - 4: $m^{(k)} \leftarrow \text{PE}(C'_k) \quad \forall k \in \{1, \dots, p\}$ ▷ Sec. 4.4
 - 5: Sample a context $h_{\text{ctx}} = h_c$ and targets $\{h_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}}$ ▷ Sec. 4.3
 - 6: $z_{\text{ctx}} \leftarrow f_{\theta}(h_{\text{ctx}})$ ▷ Sec. 4.4
 - 7: $\{z_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}} \leftarrow f_{\bar{\theta}}(\{h_{\text{tgt}}^{(k)}\}_{k \in \mathcal{T}})$ ▷ Sec. 4.4
 - 8: $\hat{z}_{\text{tgt}}^{(k)} \leftarrow g_{\phi}(z_{\text{ctx}} + m^{(k)}) \quad \forall k \in \mathcal{T}$ ▷ Sec. 4.4
 - 9: $\mathcal{L}_{\text{JEPA}} \leftarrow \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} \ell_{\text{smooth-L1}}(\hat{z}_{\text{tgt}}^{(k)}, \text{sg}(z_{\text{tgt}}^{(k)}))$ ▷ Sec. 4.5
 - 10: $(\theta_{\text{HGT}}, \theta, \phi) \leftarrow \text{GRADSTEP}(\mathcal{L}_{\text{JEPA}})$
 - 11: $\bar{\theta} \leftarrow \alpha \bar{\theta} + (1 - \alpha) \theta$ ▷ Sec. 3.3
-

4.6. Downstream malware family classification

Once pretraining concludes, the inference protocols operate on a graph-level representation $z_G \in \mathbb{R}^d$ obtained by applying the target encoder $f_{\bar{\theta}}$ to the full set of subgraph embeddings produced by the HGT for graph G_n , $\{h_1, \dots, h_p\}$, and averaging the resulting latent representations as $z_G = \frac{1}{p} \sum_{k=1}^p f_{\bar{\theta}}(h_k)$. Unlike during pretraining, where $f_{\bar{\theta}}$ processes only the $|\mathcal{T}|$ target subgraphs sampled at each step, at inference time it contextualizes all p regions of the partition jointly, yielding a global representation of the behavior graph built from dependencies distributed across heterogeneous subgraphs. The averaging then condenses these distributed regularities into a fixed-size representation that is invariant to the number of communities induced in each sample.

Linear probe. We apply the linear evaluation protocol [45] as an indicator of the linear separability of the representations induced by pretraining. After the self-supervised stage, we freeze the parameters of the HGT and of the target encoder $(\theta_{\text{HGT}}, \bar{\theta})$, and we train a linear classifier on top of z_G to discriminate malware families.

Fine-tuning. We apply the fine-tuning protocol to measure the supervised adaptability of the pretrained representations, that is, the extent to which the behavioral regularities learned during pretraining can be specialized to the taxonomic objective of family classification. After the self-supervised stage, we unfreeze $(\theta_{\text{HGT}}, \bar{\theta})$ and jointly train them with an MLP classifier on the supervised family-classification objective.

Both protocols are evaluated in Section 5 under different labeled fractions of the dataset, quantifying both the separability induced by pretraining and the label efficiency of the learned representations.

5. Experiments and Analyses

5.1. Dataset and evaluation protocol

We evaluate Malware-JEPA on ACT-KingKong [46], a public collection of malware samples gathered from VirusShare² between March and October 2019 and released as per-sample XML files containing the execution traces produced by dynamic sandbox analysis.³ In this work, these traces are turned into heterogeneous behavior graphs following the construction described in Sec. 4.2. The collection exhibits a long-tail distribution consistent with the natural imbalance among malware families.

Because a sizable fraction of the public release lacks a consistent family label, we reconstruct labels by querying the MD5 hashes against VirusTotal⁴ and adopting the Microsoft Defender nomenclature under the CARO scheme [47]. After discarding samples with execution errors or no valid label, 8,218 samples distributed across 510 families are retained. To guarantee a sufficient number of training examples per family in the lowest-label regime ($\lambda_{\min} = 10\%$), we retain only families whose number of samples N_f satisfies $N_f \cdot (1 - \nu) \cdot \lambda_{\min} \geq \tau_{\min}$, where $\nu = 0.10$ is the fraction reserved for testing and $\tau_{\min} = 5$ is the minimum number of training examples per family. This criterion retains 5,118 samples across $C = 37$ families.

We reserve 10% of the samples of each retained family as a fixed universal test set, yielding 512 test samples and 4,606 samples for training and validation. On top of this split, we evaluate five labeling fractions $\lambda \in \{10\%, 20\%, 40\%, 70\%, 100\%\}$, constructed by nested stratified sampling and replicated over three independent seeds. The self-supervised pretraining stage uses all 8,218 samples with valid graphs and does not access family labels; the downstream evaluation uses exclusively the labeled subsets derived from the 5,118 retained samples.

We report Macro F1 as the primary metric because of the class imbalance in the dataset [48]. Macro F1 averages performance uniformly across the $C = 37$ families, preventing majority families from dominating the evaluation protocol [49].

5.2. Experimental setup

The four experimental regimes—JEPA pretraining, *linear probe*, *fine-tuning*, and supervised *end-to-end*—share the same HGT encoder with $L = 3$ layers, $H = 8$ attention heads, embedding dimension $d = 256$ and dropout 0.2. All regimes train on mini-batches of size 32 and use Macro F1 on the validation split to select the best checkpoint.

The regimes differ in the training objective and the terminal head. JEPA pretraining optimizes the latent-space predictive loss of Sec. 4.5 through the target encoder f_{θ} , implemented as a Transformer with 4 layers and 4 attention heads, and the predictor g_{ϕ} , a two-layer MLP with GELU activation deliberately kept narrower than the encoders ($d \rightarrow d/2 \rightarrow d$) to avoid trivial-collapse solutions, following the design rationale of Graph-JEPA [30]. The *linear probe* trains a logistic-regression classifier on top of frozen embeddings, while *fine-tuning* and supervised *end-to-end* use a two-layer MLP classifier ($256 \rightarrow 128 \rightarrow C$) with ReLU activation and dropout 0.2.

²<https://virusshare.com/>

³<https://github.com/yidun1027/ACT-KingKong>

⁴<https://www.virustotal.com/>

JEPA pretraining uses $K = 4$ target subgraphs per context, an EMA coefficient $\alpha = 0.996$ for the target encoder, and a Smooth L1 loss with $\beta = 1.0$. We run an initialization phase of $E_0 = 10$ epochs followed by an adaptive phase of 60 epochs, optimizing with AdamW at learning rate 10^{-4} and a ReduceLROnPlateau scheduler.

The *linear probe* follows the linear-evaluation protocol of [45]: the pretrained encoder is frozen, embeddings are L_2 -normalized, and a logistic-regression classifier is trained with L-BFGS by sweeping twenty values of the inverse regularization C logarithmically spaced in $[10^{-6}, 10^5]$ and reporting the best result.

The *fine-tuning* regime jointly adjusts the encoder and the MLP head with AdamW. The MLP head is warmed up for 50 epochs over frozen embeddings before joint training proceeds for another 50 epochs with cosine-annealing schedule, layer-wise learning rate decay (LLRD) with factor 0.9 across the three HGT layers to mitigate catastrophic forgetting [50], and differentiated learning rates of 5×10^{-4} for the encoder and 10^{-4} for the head. The supervised end-to-end baseline is trained from random initialization with a single global learning rate of 5×10^{-4} and cosine annealing down to 10^{-5} ; neither LLRD nor a backbone/head separation is applied, since no pretrained weights need to be protected.

5.3. Label efficiency under linear probing and fine-tuning

Figure 2 reports the label-efficiency curve over the five labeling fractions $\lambda \in \{10\%, 20\%, 40\%, 70\%, 100\%\}$ for the three evaluated regimes: linear probing with $K = 4$, fine-tuning of the pretrained model with $K = 4$, and the supervised end-to-end baseline. The JEPA pretraining stage of Sec. 4.5 is the prerequisite that produces the representations used by linear probing and fine-tuning, and therefore does not appear as a point on the curve. Table 2 reports the corresponding numerical values as mean and standard deviation across three independent seeds.

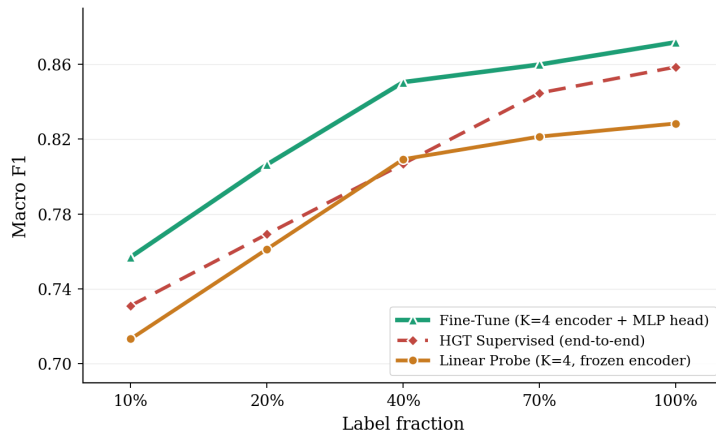


Figure 2: Label-efficiency curve. Macro F1 on the universal test set as a function of the labeling fraction λ , for linear probing with $K = 4$, the supervised end-to-end baseline, and fine-tuning with $K = 4$.

Table 2: Macro F1 ($\mu \pm \sigma$ across three seeds) by labeling fraction for linear probing with $K = 4$, the supervised end-to-end baseline, and fine-tuning with $K = 4$.

λ	LP ($K = 4$)	Supervised E2E	Fine-Tune ($K = 4$)
10%	0.7133 ± 0.0130	0.7311 ± 0.0065	0.7571 ± 0.0083
20%	0.7613 ± 0.0049	0.7693 ± 0.0090	0.8065 ± 0.0095
40%	0.8093 ± 0.0092	0.8069 ± 0.0049	0.8505 ± 0.0082
70%	0.8214 ± 0.0065	0.8447 ± 0.0101	0.8599 ± 0.0049
100%	0.8284 ± 0.0000	0.8586 ± 0.0090	0.8718 ± 0.0027

The linear probe with $K = 4$ matches the supervised baseline at $\lambda = 40\%$, where both regimes reach Macro F1 around 0.81 within one standard deviation of each other. This is direct evidence that the representation induced by JEPA pretraining is linearly separable to the same degree of family discrimination achieved by an HGT trained end-to-end on 40% of the labels. Beyond this point, the linear probe plateaus around 0.82 while the supervised baseline continues to grow up to 0.86 at $\lambda = 100\%$. This gap is a signature of the limited capacity of the linear classifier rather than a limitation of the pretrained encoder: a linear head cannot absorb the additional supervision that an encoder trained jointly with a non-linear head can. Fine-tuning closes that gap, as the next finding shows.

Fine-tuning consistently improves on the supervised baseline across all labeling fractions, with the absolute gain peaking at $\lambda = 40\%$ (+4.4 pp) and contracting in the data-rich regimes (+1.5 pp at $\lambda = 70\%$ and +1.3 pp at $\lambda = 100\%$). The most operationally relevant comparison is at $\lambda = 70\%$: fine-tuning the pretrained encoder with 70% of the labels reaches a Macro F1 of 0.8599, statistically equivalent to the 0.8586 obtained by the supervised baseline at $\lambda = 100\%$. In other words, JEPA pretraining enables matching the fully supervised model with a 30% reduction in labeling cost—the operational payoff of the label-scarcity regime that motivated this work.

The persistence of a fine-tuning advantage even at full supervision deserves attention. At $\lambda = 100\%$, the gap of +1.3 pp (0.8718 vs. 0.8586) exceeds the combined standard deviations of both regimes ($0.0027 + 0.0090 = 0.0117$), suggesting that the gain is not attributable to seed-to-seed noise. We interpret this finding as consistent with the central hypothesis of the paper: JEPA pretraining captures structural and relational regularities of malware behavior—induced by predicting the latent representations of behaviorally coherent subgraphs—that are not fully recovered by the supervised classification signal alone, even when the latter has access to the entire labeled set.

5.4. Sensitivity to the number of targets

To probe how the choice of K affects the quality of the learned representations, we evaluate four configurations $K \in \{2, 3, 4, 5\}$ under the linear-probe protocol, replicated across the five labeling fractions and three seeds. Figure 3 summarizes the aggregate behavior and Table 3 reports the per-fraction values.

The curve over K is unimodal, with $K = 4$ as the row maximum in three of the five labeling fractions and within one standard deviation of the best result in the other two. At the lower end, $K = 2$ underperforms every other configuration at every labeling fraction by a clear margin, indicating that too few targets are not enough to drive the pretraining signal. At the upper end, $K = 5$ drops back to a regime comparable to $K = 2$, in contrast with the plateau on the $K \in \{3, 4\}$

side—revealing that the deterioration past the optimum is sharp rather than gradual. At $\lambda = 10\%$, the configurations $K \in \{3, 4, 5\}$ are indistinguishable within their standard deviations.

We interpret this behavior as a trade-off internal to the JEPA pretext: the single context representation z_{ctx} must condense enough information to predict all $|\mathcal{T}|$ target embeddings from the same forward pass. Small K limits the diversity of structural regions that the predictor sees per step, while large K forces z_{ctx} to support more simultaneous predictions whose gradient signals may interfere, hindering the induction of stable representations. The sharp drop observed between $K = 4$ and $K = 5$ is consistent with this reading: once the joint-prediction load surpasses the capacity that a single context vector can support, performance falls quickly rather than degrading smoothly. We adopt $K = 4$ in the main experiments of Table 2 as the configuration that maximizes performance across most labeling fractions without entering this regime.

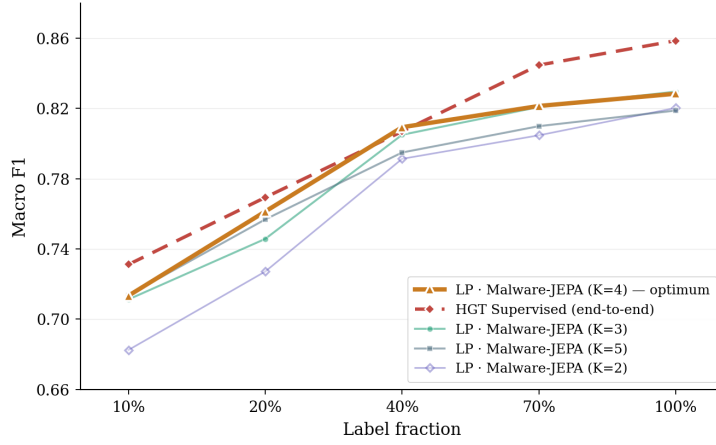


Figure 3: Sensitivity to the number of targets K . Macro F1 (linear probe) as a function of the number of target subgraphs per context K , aggregated across the five labeling fractions by arithmetic mean.

Table 3: Macro F1 ($\mu \pm \sigma$ across three seeds) by labeling fraction for linear probing with $K \in \{2, 3, 4, 5\}$ and the supervised baseline. The bold cell is the row maximum.

λ	LP $K = 2$	LP $K = 3$	LP $K = 4$	LP $K = 5$	Supervised
10%	0.6824 ± 0.0114	0.7112 ± 0.0090	0.7133 ± 0.0130	0.7136 ± 0.0086	0.7311 ± 0.0065
20%	0.7271 ± 0.0081	0.7457 ± 0.0161	0.7613 ± 0.0049	0.7568 ± 0.0138	0.7693 ± 0.0090
40%	0.7913 ± 0.0112	0.8050 ± 0.0085	0.8093 ± 0.0092	0.7949 ± 0.0075	0.8069 ± 0.0049
70%	0.8047 ± 0.0123	0.8208 ± 0.0064	0.8214 ± 0.0065	0.8099 ± 0.0022	0.8447 ± 0.0101
100%	0.8203 ± 0.0116	0.8296 ± 0.0000	0.8284 ± 0.0000	0.8188 ± 0.0000	0.8586 ± 0.0090

6. Conclusion and Future Work

This work introduces Malware-JEPA, the first extension of the Joint-Embedding Predictive Architecture paradigm to self-supervised learning on heterogeneous behavior graphs for malware family classification. The method combines HGT-based heterogeneous encoding, attention-guided adaptive partitioning, and latent-space prediction between context and target subgraphs,

avoiding both generative reconstruction of execution events and contrastive sampling that would distort the typed relational semantics of the graph.

Three findings on ACT-KingKong substantiate this proposal. A linear probe on the representations induced by Malware-JEPA matches the fully supervised HGT baseline when both are trained with only 40% of the labels, evidencing that latent-space prediction over heterogeneous subgraphs already yields linearly separable structure for family discrimination. Fine-tuning the pretrained encoder reaches the fully supervised baseline trained on the complete labeled set with 70% of the labels—a 30% reduction in labeling cost without sacrificing performance. Moreover, fine-tuning retains an advantage over the supervised baseline even when both have access to the full label set, suggesting that the predictive pretraining objective captures structural and relational regularities of malware behavior that the supervised signal alone does not fully recover.

Taken together, these results position predictive learning in latent space as a viable path beyond the supervision bottleneck that motivates malware family classification. By predicting the structure of behavior rather than reconstructing it, Malware-JEPA opens a research direction whose scope extends past malware: predictive latent learning on heterogeneous graphs with rich relational semantics, wherever labeled data is scarce and the regularities of interest live in how typed interactions relate to one another.

6.1. Limitations

We acknowledge four primary limitations of this work. First, the experimental characterization focuses on the effect of the number of targets K , and we do not report systematic ablations of the remaining components that define the JEPA architecture in Malware-JEPA. Second, the evaluation is conducted on a single dataset, ACT-KingKong, so the generalization of the method to other malware datasets is not established. Third, we compare Malware-JEPA only against a fully supervised HGT baseline; no comparison against other self-supervised methods on graphs, whether reconstructive or contrastive, is reported. Fourth, the analysis is presented through aggregated classification metrics and does not include a qualitative characterization of the communities induced by Att-LPA or of the organization of the learned latent space.

6.2. Future work

Each of the limitations declared above suggests a natural direction for future work. Systematic ablation studies of the components that define the JEPA architecture in Malware-JEPA would isolate their individual contributions to the quality of the learned representations and help identify which of them most affect the downstream performance on malware family classification. Validation on additional malware datasets, with their own family taxonomies and execution-trace conventions, would establish the extent to which the findings reported here transfer beyond ACT-KingKong. Comparison against other self-supervised methods on graphs—reconstructive baselines such as GraphMAE and contrastive baselines adapted to heterogeneous behavior graphs—would position Malware-JEPA within the broader landscape of self-supervised graph learning. A qualitative characterization of the communities induced by Att-LPA and of the organization of the learned latent space would, in addition, provide interpretability mechanisms over the structural and relational regularities that the predictive pretraining captures.

Beyond these defensive extensions, a broader research direction opens naturally: applying the predictive latent paradigm to other domains of heterogeneous graphs with rich relational

semantics—knowledge graphs, biological networks, or social structures—would evaluate the extent to which latent-space prediction over heterogeneous subgraphs constitutes a general principle of self-supervised learning on relational data, of which malware family classification is the first instantiation.

Author Contributions

Andrés David Montenegro Alvarado: *Conceptualization* (proposed the research problem of label-efficient malware family classification and the framing of self-supervised pretraining over heterogeneous behavior graphs); *Methodology* (designed the Malware-JEPA architecture, including the Att-LPA partitioning, the context–target sampling, the asymmetric encoder–predictor design, and the latent-space training objective); *Software* (implemented the complete experimental pipeline); *Investigation* (executed the experimental campaign on ACT-KingKong, including label reconstruction, dataset partitioning across labeling fractions, and replication over independent seeds); *Formal Analysis* (analyzed the label-efficiency results and the sensitivity to the number of targets K); *Visualization* (produced the figures and tables of the experimental section and the architectural overview); *Writing – Original Draft* (wrote the manuscript in its entirety).

Nelson Enrique Vera Parra: *Supervision* (oversaw the research project and provided strategic direction throughout the methodological design and the experimental campaign); *Project Administration* (managed the research project across its successive iterations); *Conceptualization* (contributed to the framing of the research problem and to scope decisions on Malware-JEPA as a methodological–formulative contribution); *Writing – Review & Editing* (reviewed and refined the manuscript for precision of the technical exposition and coherence of the narrative).

References

- [1] A. Grégio, R. Bonacin, A. C. De Marchi, O. F. Nabuco, P. L. De Geus, An ontology of suspicious software behavior, *Applied Ontology* 11 (1) (2016) 29–49. doi:10.3233/AO-160163.
- [2] A. R. A. Grégio, V. M. Afonso, D. S. o. Fernandes Filho, P. L. De Geus, M. Jino, Toward a taxonomy of malware behaviors, *The Computer Journal* 58 (10) (2015) 2758–2777. doi:10.1093/comjnl/bxv047.
- [3] C. Yavvari, A. Tokhtabayev, H. Rangwala, A. Stavrou, Malware characterization using behavioral components, in: *Computer Network Security: 6th International Conference on Mathematical Methods, Models and Architectures for Computer Network Security (MMM-ACNS)*, Vol. 7531 of *Lecture Notes in Computer Science*, Springer, 2012, pp. 226–239. doi:10.1007/978-3-642-33704-8_20.
- [4] AV-TEST Institute, Malware statistics and trends report, <https://www.av-test.org/en/statistics/malware/>, accessed: 8 May 2026. Cited figure: approximately 450,000 new malware samples per day, average of 2024–2025. (2026).
- [5] M. G. Gaber, M. Ahmed, H. Janicke, Malware detection with artificial intelligence: A systematic literature review, *ACM Computing Surveys* 56 (6) (2024) 1–33. doi:10.1145/3638552.
- [6] P. Maniriho, A. N. Mahmood, M. J. M. Chowdhury, A study on malicious software behaviour analysis and detection techniques: Taxonomy, current trends and challenges, *Future Generation Computer Systems* 130 (2022) 1–18. doi:10.1016/j.future.2021.11.030.
- [7] R. J. Joyce, D. Amlani, C. Nicholas, E. Raff, MOTIF: A malware reference dataset with ground truth family labels, *Computers and Security* 124 (2023) 102921. doi:10.1016/j.cose.2022.102921.
- [8] R. J. Joyce, E. Raff, C. Nicholas, A framework for cluster and classifier evaluation in the absence of reference labels, in: *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*, ACM, 2021, pp. 73–84. doi:10.1145/3474369.3486867.
- [9] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proceedings of The Web Conference 2020*, Association for Computing Machinery, 2020, pp. 2704–2710. doi:10.1145/3366423.3380027.

- [10] C. Zhang, D. Song, C. Huang, A. Swami, N. V. Chawla, Heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, 2019, pp. 793–803. doi:10.1145/3292500.3330961.
- [11] X. Wang, H. Ji, C. Shi, B. Wang, P. Cui, P. S. Yu, Y. Ye, Heterogeneous graph attention network, in: Proceedings of The World Wide Web Conference, Association for Computing Machinery, 2019, pp. 2022–2032. doi:10.1145/3308558.3313562.
- [12] C. Liu, B. Li, J. Zhao, M. Su, X. D. Liu, MG-DVD: A real-time framework for malware variant detection based on dynamic heterogeneous graph learning, in: Proceedings of the 30th International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence, 2021, pp. 1512–1519. doi:10.24963/ijcai.2021/209.
- [13] Y. Fan, S. Hou, Y. Zhang, Y. Ye, M. Abdulhayoglu, Gotcha – sly malware! Scorpion: A metagraph2vec based malware detection system, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, 2018, pp. 253–262. doi:10.1145/3219819.3219862.
- [14] S. Wang, Z. Chen, X. Yu, D. Li, J. Ni, L.-A. Tang, J. Gui, Z. Li, H. Chen, P. S. Yu, Heterogeneous graph matching networks for unknown malware detection, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence, 2019, pp. 3762–3770. doi:10.24963/ijcai.2019/522.
- [15] D. Votipka, S. M. Rabin, K. Micinski, J. S. Foster, M. M. Mazurek, An observational investigation of reverse engineers’ processes, in: Proceedings of the 29th USENIX Security Symposium, USENIX Association, 2020.
- [16] X. Ugarte-Pedrero, M. Graziano, D. Balzarotti, A close look at a daily dataset of malware samples, *ACM Transactions on Privacy and Security* 22 (1) (2018) 1–30. doi:10.1145/3291061.
- [17] J. Li, R. Wu, W. Sun, L. Chen, S. Tian, L. Zhu, C. Meng, Z. Zheng, W. Wang, What’s behind the mask: Understanding masked graph modeling for graph autoencoders, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, 2023, pp. 1268–1279. doi:10.1145/3580305.3599546.
- [18] Z. Hou, Y. He, Y. Cen, X. Liu, Y. Dong, E. Kharlamov, J. Tang, GraphMAE2: A decoding-enhanced masked self-supervised graph learner, in: Proceedings of the ACM Web Conference 2023, Association for Computing Machinery, 2023, pp. 737–746. doi:10.1145/3543507.3583379.
- [19] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, J. Tang, GraphMAE: Self-supervised masked graph autoencoders, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, 2022, pp. 594–604. doi:10.1145/3534678.3539321.
- [20] Q. Tan, N. Liu, X. Huang, S.-H. Choi, L. Li, R. Chen, X. Hu, S2GAE: Self-supervised graph autoencoders are generalizable learners with graph masking, in: Proceedings of the 16th ACM International Conference on Web Search and Data Mining, Association for Computing Machinery, 2023, pp. 787–795. doi:10.1145/3539597.3570404.
- [21] Z. Zhao, Y. Su, Y. Li, Y. Zou, R. Li, R. Zhang, A survey on self-supervised graph foundation models: Knowledge-based perspective, *IEEE Transactions on Knowledge and Data Engineering* 37 (8) (2025) 4389–4410. doi:10.1109/TKDE.2025.3568147.
- [22] Y. Liu, M. Jin, S. Pan, C. Zhou, Y. Zheng, F. Xia, P. S. Yu, Graph self-supervised learning: A survey, *IEEE Transactions on Knowledge and Data Engineering* 35 (6) (2023) 5879–5900. doi:10.1109/TKDE.2022.3172903.
- [23] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, Y. Shen, Graph contrastive learning with augmentations, in: *Advances in Neural Information Processing Systems* 33, Curran Associates, 2020.
- [24] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, *arXiv preprint arXiv:2006.04131* (2020).
- [25] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, R. D. Hjelm, Deep graph infomax, in: *7th International Conference on Learning Representations*, 2019.
- [26] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Graph contrastive learning with adaptive augmentation, in: *Proceedings of the Web Conference 2021*, Association for Computing Machinery, 2021, pp. 2069–2080. doi:10.1145/3442381.3449802.
- [27] Y. Zhou, Y. Cao, Y. Liu, Y. Shang, P. Zhang, Z. Lin, Y. Yue, B. Wang, X. Fu, W. Wang, Multi-aspect heterogeneous graph augmentation, in: *Proceedings of the ACM Web Conference 2023*, Association for Computing Machinery, 2023, pp. 39–48. doi:10.1145/3543507.3583208.
- [28] Y. LeCun, A path towards autonomous machine intelligence, *OpenReview* (2022). URL <https://openreview.net/forum?id=BZ5a1r-kVsf>
- [29] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, N. Ballas, Self-supervised learning from images with a joint-embedding predictive architecture, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15619–15629. doi:10.1109/CVPR52729.2023.01499.
- [30] G. Skenderi, H. Li, J. Tang, M. Cristani, Graph-level representation learning with joint-embedding predictive architectures, *Transactions on Machine Learning Research* (2025).
- [31] H. Gunduz, Malware detection framework based on graph variational autoencoder extracted embeddings from

- API-call graphs, *PeerJ Computer Science* 8 (2022). doi:10.7717/peerj-cs.988.
- [32] F. Deldar, M. Abadi, M. Ebrahimifard, Android malware detection using supervised deep graph representation learning, in: *Proceedings of the 12th International Conference on Computer and Knowledge Engineering (ICCKE)*, IEEE, 2022, pp. 348–354. doi:10.1109/ICCKE57176.2022.9960076.
 - [33] T. Wang, M. Liu, H. Li, L. Zhao, C. Jiang, C. Xia, B. Cui, ArchSentry: Enhanced Android malware detection via hierarchical semantic extraction, *IEEE Transactions on Network and Service Management* 22 (3) (2025) 2822–2837. doi:10.1109/TNSM.2025.3559255.
 - [34] P. Wang, H. C. Li, H. C. Lin, W. H. Lin, N. Z. Xie, A transductive zero-shot learning framework for ransomware detection using malware knowledge graphs, *Information* 16 (6) (2025). doi:10.3390/info16060458.
 - [35] M. Gao, P. Wu, L. Pan, Malware detection with limited supervised information via contrastive learning on API call sequences, in: *Information and Communications Security (ICICS)*, Vol. 13407 of *Lecture Notes in Computer Science*, Springer, 2022, pp. 492–507. doi:10.1007/978-3-031-15777-6_27.
 - [36] M. Gao, P. Wu, L. Pan, MINES: Multi-perspective API call sequence behavior fusion malware classification 14853 (2024) 210–220. doi:10.1007/978-981-97-5562-2_13.
 - [37] C. Liu, B. Li, J. Zhao, Z. Zhen, X. Liu, Q. Zhang, FewM-HGCL: Few-shot malware variants detection via heterogeneous graph contrastive learning, *IEEE Transactions on Dependable and Secure Computing* (2022). doi:10.1109/TDSC.2022.3216902.
 - [38] T. Han, C. Xu, D. Zhan, T. Wang, Z. Pan, et al., SHIELD: Semantic-guided graph contrastive learning for malware detection, *Expert Systems with Applications* 308 (2026) 131108. doi:10.1016/j.eswa.2026.131108.
 - [39] F. Piccoli, G. Vogel, J. M. Weber, Joint embedding predictive architecture for self-supervised pretraining on polymer molecular graphs, *Digital Discovery* (2026). doi:10.1039/d5dd00308c.
 - [40] C. Liu, B. Li, X. Liu, C. Li, J. Bao, Evolving malware detection through instant dynamic graph inverse reinforcement learning, *Knowledge-Based Systems* 299 (2024). doi:10.1016/j.knosys.2024.111991.
 - [41] C. Liu, B. Li, Y. Wu, X. Liu, J. Li, C. Li, DynMD: Energy-based dynamic graph representation learning for malware detection, *IEEE Transactions on Dependable and Secure Computing* (2025). doi:10.1109/TDSC.2025.3619926.
 - [42] Y. Yang, Z. Guan, Z. Wang, W. Zhao, C. Xu, W. Lu, J. Huang, Self-supervised heterogeneous graph pre-training based on structural clustering, in: *Advances in Neural Information Processing Systems 35 (NeurIPS)*, Curran Associates, 2022.
 - [43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Curran Associates, 2017, pp. 6000–6010.
 - [44] U. N. Raghavan, R. Albert, S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, *Physical Review E* 76 (3) (2007) 036106. doi:10.1103/PhysRevE.76.036106.
 - [45] T. Chen, S. Kornblith, M. Norouzi, G. Hinton, A simple framework for contrastive learning of visual representations, in: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, PMLR, 2020.
 - [46] C. Liu, B. Li, J. Zhao, W. Feng, X. Liu, C. Li, A2-CLM: Few-shot malware detection based on adversarial heterogeneous graph augmentation, *IEEE Transactions on Information Forensics and Security* 19 (2024) 2023–2038. doi:10.1109/TIFS.2023.3345640.
 - [47] V. Bontchev, F. Skulason, A. Solomon, CARO virus naming convention (1991).
 - [48] M. Sokolova, G. Lapalme, A systematic analysis of performance measures for classification tasks, *Information Processing & Management* 45 (4) (2009) 427–437. doi:10.1016/j.ipm.2009.03.002.
 - [49] Z. C. Lipton, C. Elkan, B. Naryanaswamy, Optimal thresholding of classifiers to maximize F1 measure, in: *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, Vol. 8725 of *Lecture Notes in Computer Science*, Springer, 2014, pp. 225–239. doi:10.1007/978-3-662-44851-9_15.
 - [50] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, R. Hadsell, Overcoming catastrophic forgetting in neural networks, *Proceedings of the National Academy of Sciences* 114 (13) (2017) 3521–3526. doi:10.1073/pnas.1611835114.